The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software
Unit Coverage: 1-5
Student Name: Aman Dwivedi
Registration Number: 24BCE10573
Slot: B22
Date of Submission: 31st March, 2026

1. Introduction

The Apache HTTP Server is a fantastic, no-cost, and open-source web server that works across all your favorite platforms. It's available under the friendly terms of the Apache License 2.0. A passionate bunch of developers, all part of the Apache Software Foundation, are the ones who lovingly build and keep it running smoothly.

Most Apache HTTP Servers are humming along happily on Linux. But hey, it's not exclusive! You'll also find the latest versions making a home on Microsoft Windows, OpenVMS, and a bunch of other systems that feel like Unix. If you look back, it used to pop up on things like NetWare, OS/2, and even had versions tailored for big mainframes. It's gotten around!

The story of Apache began back in early 1995. It was built upon the foundations of the older NCSA HTTPd server, but when development on that original code slowed down, a new effort started. Apache really took off, becoming a crucial player in the early days of the World Wide Web, quickly becoming the most popular HTTP server and surpassing its predecessor. It hit a major milestone in 2009, proudly becoming the first web server to power over 100 million websites.

Based on Netcraft's March 2025 estimates for the million busiest websites, Cloudflare was leading the pack at 22.99%, followed closely by Nginx at 20.11%. Apache came in third at 17.83%, and Microsoft Internet Information Services lagged behind at 4.16%.

However, when looking at all websites according to W3Techs in April 2025, Nginx was the clear winner, powering 33.8% of sites. Apache was still a strong contender, holding 26.4%, while Cloudflare Server was just behind with 23.5%.

2. Choosing Your Software

For this audit, I have selected the following software from the approved list:

- **Software:** Apache HTTP Server
- **Category:** Web Server
- **License:** Apache 2.0
- **Why it's interesting:** The Apache HTTP Server (commonly known simply as Apache) is considered interesting and highly valuable due to its open-source nature, robust flexibility through a modular architecture, extensive community support, and historical significance in powering the early growth of the internet.

3. Project Report Structure

Part A: Origin and Philosophy

A1. The Problem this Software was Created to Solve

The Apache HTTP Server's story really kicks off in the early days of the World Wide Web. Back then, in the early 1990s, the undisputed king of web servers was NCSA HTTPd, a brilliant piece of software created by Rob McCool over at the National Center for Supercomputing Applications. It was essentially the engine driving the early internet. But then, in 1994, McCool moved on from NCSA, and just like that, the development of the server ground to a halt.

The massive issue was that webmasters all over the world depended on this software. When things went wrong—a bug popped up, or the web just got too

big and busy for the old features to handle—there was no one left in charge to roll out updates or patch dangerous security holes. The software was basically left to rot.

A bunch of the original web trailblazers, including Brian Behlendorf, got totally fed up with this broken system. They realized waiting for a company or a main office to fix things wasn't going to work anymore. So, they started emailing code tweaks, which they called "patches," to each other to keep their own servers running.

This whole thing started with a bunch of people working together to create what they called the "Apache Group." They made a conscious choice to share their work openly, because back then, the only realistic way to keep the internet's infrastructure running was by collaborating. By putting their heads together and checking each other's code, they were able to build a web server that was far more secure and solid than anything a single company could manage on its own. The name "Apache" is a cute little inside joke about how it all began: it was literally a "patchy" server, built piece by piece from fixes and additions sent in by the community.

A2. The License: What it Actually Says

The Apache HTTP Server is available under the friendly Apache License 2.0. Think of it this way: when you use it, you get four basic superpowers from the world of free software. You're free to use the program however you want. You're free to peek under the hood, see how it works, and make changes. You're free to share copies with others. And if you make your own awesome version, you're free to share those modified copies, too! The Apache License hands you all four of these freedoms, no strings attached..

But here's the key: the Apache License is what we call "permissive," not "copyleft." Think of it this way: a company can grab the original Apache code, tweak it to fit their business perfectly, wrap it up in their own proprietary product, and then sell it for a profit—all without ever having to share their changes with the open-source community. The only things they absolutely have

to do are give a shout-out to the original creators and include a copy of the Apache License with their modified software.

This is totally different from a copyleft license, like the GPL. With the GPL, if you mess with the code and then share it, you're legally required to make your updated source code public and use that same license. Licenses like MIT and Apache are all about giving developers and companies the most flexibility to use the code however they want, while the GPL is focused on keeping the code open forever. If I were creating a software project that I wanted lots of companies to use, I'd definitely pick the Apache License. Its laid-back approach avoids the legal headaches that often stop big businesses from using copyleft software.

This difference is a big deal and it's often summed up as "free as in free beer" versus "free as in freedom." Think of "free beer" as just meaning you don't have to pay anything for the product—it's *gratis*. "Free freedom," on the other hand, means you actually have the *liberty* to control and change the software—it's *libre*. For instance, when you download the Apache server to run a website, you're getting "free beer" because you skip the licensing fees. But the real freedom is that you can peek inside its core code, tweak how it handles requests, and even recompile it to perfectly fit your unique needs. That's true liberty.

A3. The Ethics of Open Source

The idea behind open-source software brings up some really tough questions about fairness and how we should share knowledge. You could easily argue that *all* software should be open source. Think about it: when the code is out in the open, thousands of smart people can look it over, which makes it safer and stronger right away. Plus, it levels the playing field—students and people in developing countries get access to the same amazing tools as huge, rich companies.

But there's another side to this, and it comes down to money and how we pay for new ideas. Some fields are super specialized—I'm talking about things like cutting-edge medical scanning programs or big-budget video games—that need tons of cash upfront and years of high-risk effort. If we take away the chance for companies to make that money back through exclusive ownership, the funding for those specific, major breakthroughs might just dry up completely.

It's a hot topic: is it fair that huge companies make massive profits using software that was built for free by a community? Think about companies like Meta, Google, and IBM—they rely super heavily on open-source projects like the Linux kernel and the Apache web server.

At first glance, it might look like a giant corporation is just freeloading off the community's hard work. But the reality is much more of a partnership. These companies aren't just takers; they invest back into the system. They hire thousands of talented developers whose entire job is to write code and send those improvements right back to the open-source foundations they use.

In the tech world, there's a saying: "standing on the shoulders of giants." It means that today's amazing technology is only possible because of the open foundations that people before us built. Open source doesn't slow down creativity; it supercharges it! Since tools like Apache already exist, a new startup doesn't have to waste its first year building a basic web server from scratch. They can download that foundation for free and immediately jump into building the innovative stuff that sets them apart.

Part B: Linux Footprint

1. Installation Method

On Ubuntu 24.04, the Apache HTTP server is distributed through the Advanced Package Tool (APT), It does not need to be compiled from source; it is safely installed directly from Canonical’s official repositories using the apache2 package.

- **sudo apt update && sudo apt install apache2 -y**

```
rijul@rijul:~$ sudo apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-data apache2-utils
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom
The following NEW packages will be installed:
  apache2 apache2-data apache2-utils
0 upgraded, 3 newly installed, 0 to remove and 4 not upgraded.
Need to get 352 kB of archives.
After this operation, 1,677 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-data all 2.4.58-1ubuntu8.11 [163 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-utils amd64 2.4.58-1ubuntu8.11 [98.2 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2 amd64 2.4.58-1ubuntu8.11 [90.2 kB]
Fetched 352 kB in 1s (256 kB/s)
Selecting previously unselected package apache2-data.
(Reading database ... 313782 files and directories currently installed.)
Preparing to unpack .../apache2-data_2.4.58-1ubuntu8.11_all.deb ...
Unpacking apache2-data (2.4.58-1ubuntu8.11) ...
Selecting previously unselected package apache2-utils.
Preparing to unpack .../apache2-utils_2.4.58-1ubuntu8.11_amd64.deb ...
Unpacking apache2-utils (2.4.58-1ubuntu8.11) ...
Selecting previously unselected package apache2.
Preparing to unpack .../apache2_2.4.58-1ubuntu8.11_amd64.deb ...
Unpacking apache2 (2.4.58-1ubuntu8.11) ...
Setting up apache2-data (2.4.58-1ubuntu8.11) ...
Setting up apache2-utils (2.4.58-1ubuntu8.11) ...
Setting up apache2 (2.4.58-1ubuntu8.11) ...
apache-htcacheclean.service is a disabled or a static unit not running, not starting it.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for ufw (0.36.2-6) ...
rijul@rijul:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
rijul@rijul:~$
```

2. Key Directories

Following the LInux Filesystem Hierarchy Standards, Apache separates its executables, configuration files, and logs into specific directories:

- **Configuration files:** Stored in /etc/apache2/ (specifically the apache2.conf file and sites-available folder).
- **Log files:** Stored in /var/log/apache2/ (contains access.log and error.log).
- **Web Root:** Stored in /var/www/html/ (where the actual website files live).
- **Command to run for your screenshot (shows the config directory):**

  ls -l /etc/apache2/

```
rijul@rijul:~$ ls -l /etc/apache2/
total 80
-rw-r--r-- 1 root root  7178 Mar  5 23:01 apache2.conf
drwxr-xr-x 2 root root  4096 Mar 25 21:30 conf-available
drwxr-xr-x 2 root root  4096 Mar 25 21:27 conf-enabled
-rw-r--r-- 1 root root  1782 Mar 18  2024 envvars
-rw-r--r-- 1 root root 31063 Mar 18  2024 magic
drwxr-xr-x 2 root root 12288 Mar 25 21:30 mods-available
drwxr-xr-x 2 root root  4096 Mar 25 21:27 mods-enabled
-rw-r--r-- 1 root root   274 Mar 18  2024 ports.conf
drwxr-xr-x 2 root root  4096 Mar 25 21:30 sites-available
drwxr-xr-x 2 root root  4096 Mar 25 21:27 sites-enabled
rijul@rijul:~$
```

3. User and Group Security

By default, the Apache service on Ubuntu runs under the www-data user and the www-data group. It does not run as the root user. This matters for security due to the principle of least privilege. If a hacker finds a vulnerability in a website and takes over the web server, the only get the restricted permissions of the www-data user, preventing them from destroying the core Linux operating system.

- **Command to run for your screenshot:**

  ps -ef | grep apache2

```
rijul@rijul:~$ ps -ef | grep apache2
root     3648084       1  0 21:30 ?        00:00:00 /usr/sbin/apache2 -k start
www-data 3648088 3648084  0 21:30 ?        00:00:00 /usr/sbin/apache2 -k start
www-data 3648089 3648084  0 21:30 ?        00:00:00 /usr/sbin/apache2 -k start
rijul    3649189 3644171  0 21:34 pts/0    00:00:00 grep --color=auto apache2
rijul@rijul:~$
```

4. Service Management

Apache runs as a background daemon and is managed by systemd on Ubuntu 24.04. This allows the administrator to start, stop, or check the health of the web server using systemctl commands.

- **Commands to list in your report:**
  - **Start:** sudo systemctl start apache2
  - **Stop:** sudo systemctl stop apache2
  - **Restart:** sudo systemctl restart apache2

- **Command to run for your screenshot:**

  Sudo systemctl status apache2

5. Update Model

The open-source community constantly releases security patches for Apache. However, on Ubuntu 24.04, these updates are managed by the OS maintainers (Canonical). Canonical's security team backports community security patches to the specific version of Apache running on Ubuntu. The server administrator receives these patches simply by running the system's built-in package updater, ensuring stability and security without breaking the server.

- **Command to run for your screenshot:**

  sudo apt update && sudo apt --only-upgrade install apache2

Part C: The FOSS Ecosystem

1. Dependencies: What tools does Apache depend on?

Apache does not exist in a vacuum; it relies heavily on the other foundational open-source tools to function:

- **APR (Apache Portable Runtime):** The most critical dependency. APR acts as a bridge between Apache and the underlying operating system, ensuring that the web server runs the same way on Linux, Windows, or macOS without rewriting the core code.
- **PCRE (Perl Compatible Regular Expressions):** Used by Apache for advanced URL rewriting and complex configuration rules.
- **OpenSSL:** The open-source cryptography toolkit that Apache relies on to provide HTTPS encryption for secure web traffic.
- **GCC (GNU Compiler Collection):** To build Apache from source, developers rely on the standard open-source C compiler.

2. Impact: What has it enabled or inspired?

Apache is the foundation upon which much of the modern web was built.

- **Enabled:** It directly enabled the rise of open-source Content Management Systems (CMS) like WordPress, Joomla, and Drupal, which were specifically designed to be hosted on Apache.
- **Inspired (and Provoked):** Apache's early architecture struggled to handle 10,000 concurrent connections (the famous "C10k problem"). This directly inspired the creation of its biggest open-source rival, **Nginx**, which was built with a different, event-driven architecture to solve the exact scaling issues Apache faced in the 2000s.
- **Spin-offs:** The community around the server grew so successful that it spawned the **Apache Software Foundation (ASF)**, which now hosts hundreds of other massive FOSS projects like Apache Kafka, Hadoop, and Tomcat.

3. The LAMP Stack Connection

Apache is the "A" in the ubiquitous LAMP stack (Linux, Apache, MySQL, PHP/Python/Perl), which became the default architecture for web applications for over a decade.

- **Its Role:** In this stack, Apache acts as the "front door" or the middleman. When a user types a URL into a browser, Apache receives the HTTP request. If the user asks for a static image, Apache fetches it directly from the Linux filesystem. If the user asks for dynamic content (like a login page), Apache hands the request off to PHP, which queries the MySQL database, generates the HTML, hands it back to Apache, and Apache finally sends it over the internet to the user's browser.

4. Community and Governance

The Apache HTTP Server is not governed by a single corporate CEO; it operates as a strict meritocracy governed by the **Apache Software Foundation (ASF)**.

- **Governance Model:** "Those who do the work make the decisions." Developers who consistently contribute high-quality code are voted in as "Committers" and given write-access to the code repository.
- **Communication:** The project strongly favors asynchronous, text-based communication. The golden rule of the ASF is, "If it didn't happen on a mailing list, it didn't happen." They use public mailing lists (like *dev@httpd.apache.org*) for all technical decisions, ensuring a permanent, searchable history of why code was changed.
- **Conferences:** The community gathers physically and virtually at **ApacheCon** (now Community Over Code), an annual FOSS conference dedicated to projects under the ASF umbrella.

Part D: Open Source vs Proprietary (Critical Analysis)

Comparison: Apache HTTP Server vs. Microsoft IIS

Dimension	Open Source (Apache HTTP Server)	Proprietary Alternative (Microsoft IIS)
Cost	**100% Free.** There are absolutely no licensing fees, regardless of how many servers you	**Expensive.** While IIS is technically "free," it only runs on Windows Server, which requires

	deploy it on.	expensive, enterprise-level OS licensing.
Security (who can audit the code?)	**Transparent.** The source code is public. Thousands of independent developers and security researchers audit the code and submit patches daily.	**Closed Source.** Only Microsoft's internal engineers can audit the core code. Users must blindly trust Microsoft to find and patch vulnerabilities.
Support & reliability	**Community-driven.** Support comes from massive community forums, mailing lists, and documentation. Paid support is only available via 3rd-party Linux vendors (like Red Hat).	**Corporate-backed.** Users get official, dedicated, SLA-backed enterprise support directly from Microsoft.
Freedom to modify	**Total Freedom.** Administrators can rewrite core modules, recompile the server from scratch, and adapt it to any highly specific use case.	**Zero Freedom.** Administrators cannot view or alter the core code. Customizations are strictly limited to the official APIs and extensions Microsoft allows.
Community vs corporate control	**Meritocracy.** Governed by the Apache Software Foundation. Decisions are made publicly by the community members who contribute the most code.	**Corporate Monopoly.** Governed entirely by Microsoft. Features are added or deprecated based strictly on Microsoft's corporate roadmap and profit incentives.

Overall Verdict

The Apache HTTP Server? Oh yeah, I'd totally go with that one for a real-world project. Sure, the new kids like Nginx might be a hair faster when they're just

pushing simple static files, but honestly, Apache's flexibility is just unbeatable. It's got this awesome modular design, and the .htaccess files are lifesavers—they let us set up super-complex rules for a specific folder without having to reboot the whole server. Plus, it's open-source and free, which is a massive win for any startup trying to launch without a huge budget. It fits perfectly with the classic LAMP stack, making it the most trustworthy, battle-hardened "front door" for today's dynamic web applications.

And hey, I'd absolutely love to contribute to the project, too. Being part of the Apache community isn't just about coding; it's about helping keep the actual infrastructure of the internet humming. Think about it: Apache runs almost a third of all websites globally, so fixing even a tiny bug or clarifying a piece of documentation makes a huge, positive impact on millions of people around the world. It’s the ultimate expression of the open-source spirit: building something great together and then giving it away so the whole world can benefit from our collective work.

---

4. Shell Script Tasks

Script 1: System Identity Report

Script Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name - FILL THIS IN]
# Course: Open Source Software
# Purpose: Introduces the Linux system, displaying OS info, user info, and license.

# --- Variables ---
STUDENT_NAME="[Your Name - FILL THIS IN]"
SOFTWARE_CHOICE="Apache HTTP Server"

# --- System Info Gathering (Command Substitution) ---
# Extracts the kernel release version
KERNEL=$(uname -r)
# Gets the current logged-in user
USER_NAME=$(whoami)
# Gets the user's home directory from the environment variable
HOME_DIR=$HOME
# Gets the human-readable uptime of the system
UPTIME=$(uptime -p)
# Extracts the OS distribution name from the os-release file
DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d '=' -f 2 | tr -d '"')
# Gets the current date and time
CURRENT_DATE=$(date +"%A, %d %B %Y %T")
# The license governing the Linux kernel
LICENSE_MSG="The Linux Kernel is proudly covered by the GNU General Public License
v2 (GPL)."

# --- Display Output ---
echo "====================================================="
echo "   The Open Source Audit | Welcome Screen"
echo "====================================================="
echo "Student Name    : $STUDENT_NAME"
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "-----------------------------------------------"
echo "Current Date    : $CURRENT_DATE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "-----------------------------------------------"
echo "Logged-in User  : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
```

```
echo "System Uptime   : $UPTIME"
echo "-----------------------------------------------"
echo "$LICENSE_MSG"
echo "===================================================="
```

Script 2: FOSS Package Inspector

Script Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Purpose: Checks if the chosen software is installed and prints a philosophical note.
# Note: Modified from the RPM starter code to use 'dpkg', which is the standard for Ubuntu
24.04.

PACKAGE="apache2" # 'apache2' is the package name for Apache on Ubuntu

echo "Checking system for $PACKAGE..."

# --- Check if package is installed ---
# dpkg -l lists packages. grep -q runs quietly and returns success if found.
if dpkg -l | grep -qw "^ii.*$PACKAGE"; then
    echo -e "\n[SUCCESS] $PACKAGE is installed on this system."
```

```
    echo "--- Package Details ---"
    # Extract just the Version and Description lines for clean output
    dpkg -s $PACKAGE | grep -E '^Version|^Description'
else
    echo -e "\n[WARNING] $PACKAGE is NOT installed."
    echo "You can install it by running: sudo apt install $PACKAGE"
fi

echo -e "\n--- Open Source Philosophy Note ---"
# --- Case statement to evaluate the package name ---
case $PACKAGE in
    apache2|httpd)
        echo "Apache: The web server that built the open internet and powers the LAMP
stack."
        ;;
    mysql)
        echo "MySQL: Open source at the heart of millions of dynamic web apps."
        ;;
    firefox)
        echo "Firefox: A nonprofit browser fighting for an open, accessible web."
        ;;
    vlc)
        echo "VLC: Plays anything - built by students in Paris, completely free."
        ;;
    *)
        echo "Philosophy unknown for this package. Research its FOSS origins!"
        ;;
esac
echo "----------------------------------"
```

Script 3: Disk and Permission Auditor

Script Code:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Purpose: Loops through critical system directories to report their sizes, owners, and
permissions.

# Array containing standard Linux directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "================================================"
echo "          Directory Audit Report"
echo "================================================"

# --- For loop to iterate through the array ---
for DIR in "${DIRS[@]}"; do
   # Check if the directory actually exists
   if [ -d "$DIR" ]; then
      # Extract permissions, user owner, and group owner using ls and awk
      PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
      # Extract human-readable size, suppressing permission denied errors
      SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

      # Display formatted results
      echo "[EXISTS] $DIR => Permissions/Owner: $PERMS | Size: $SIZE"
   else
      echo "[MISSING] $DIR does not exist on this system."
   fi
done

echo "-----------------------------------------------"
echo "          Specific Software Audit: Apache"
echo "-----------------------------------------------"

# --- Specific check for the Apache configuration directory ---
APACHE_DIR="/etc/apache2"

if [ -d "$APACHE_DIR" ]; then
   APACHE_PERMS=$(ls -ld "$APACHE_DIR" | awk '{print $1, $3, $4}')
   echo "[FOUND] $APACHE_DIR"
   echo "Permissions/Owner: $APACHE_PERMS"
else
```

```
    echo "[NOT FOUND] Apache config directory ($APACHE_DIR) does not exist."
fi
echo "==================================================="
```

Script 4: Log File Analyzer
Script Code:

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./script4.sh /var/log/apache2/error.log [keyword]
# Purpose: Reads a log file, counts occurrences of a keyword, and shows the latest
matches.

# Take log file path from 1st command line argument
LOGFILE=$1
# Take keyword from 2nd argument, default to "error" if not provided
KEYWORD=${2:-"error"}
COUNT=0

# --- Do-While style retry loop ---
# This simulates a do-while loop in bash to ensure we have a valid file
while true; do
    # Check if the file exists AND has a size greater than zero (-s flag)
```

```
    if [ -s "$LOGFILE" ]; then
        echo "Valid log file detected: $LOGFILE"
        break # Exit the loop, file is good
    else
        echo "Error: File '$LOGFILE' is either empty, not found, or no file was provided."
        # Prompt user to try again
        read -p "Please enter a valid log file path (e.g., /var/log/syslog): " LOGFILE
    fi
done

echo "Scanning $LOGFILE for the keyword: '$KEYWORD'..."

# --- While-Read loop ---
# Reads the file line by line
while IFS= read -r LINE; do
    # Check if the current line contains the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        # Increment the counter
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# Print the summary
echo "==================================================="
echo "SUMMARY: Keyword '$KEYWORD' found $COUNT times."
echo "==================================================="

# --- Print the last 5 matching lines ---
if [ $COUNT -gt 0 ]; then
    echo "--- Last 5 Matching Lines ---"
    # grep finds the words, tail -n 5 limits it to the bottom 5 results
    grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
fi
```

Script 5: Open Source Manifesto Generator

Script Code:

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Purpose: Interactively asks the user questions and generates a text file manifesto.

# --- Concept Demonstration: Aliases ---
# An alias is a shortcut for a command. If we uncommented the line below,
# the user could type 'manifesto' in their terminal instead of './script5.sh'
# alias manifesto='./script5.sh'

echo "==================================================="
echo "      The Open Source Manifesto Generator"
echo "==================================================="
echo "Please answer these three questions:"
echo ""

# --- Interactive Prompts ---
# -p flag allows us to prompt the user and read their input into a variable on the same line
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one software tool you would like to build and share freely: " BUILD

# Get today's date in 'Day Month Year' format
DATE=$(date +'%d %B %Y')
# Generate a dynamic output filename based on the current user
OUTPUT="manifesto_$(whoami).txt"

echo "-----------------------------------------------"
echo "Drafting your personal manifesto..."

# --- Writing to a file ---
# Using > completely overwrites the file (or creates it) for the first line
echo "My Open Source Manifesto" > "$OUTPUT"
echo "Recorded on: $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Using >> appends to the file without destroying the previous lines
echo "On this day, I declare that my daily digital workflow relies heavily on the open-source
tool $TOOL." >> "$OUTPUT"
echo "To me, software freedom is not just about cost; it ultimately means $FREEDOM." >>
"$OUTPUT"
echo "Because I benefit immensely from the collective labor of the open-source community,"
>> "$OUTPUT"
echo "I promise that one day I will build a $BUILD and share its source code freely with the
world." >> "$OUTPUT"
```

```
echo "[SUCCESS] Manifesto saved to $OUTPUT"
echo "==================================================="
echo "          Here is what you wrote:"
echo "==================================================="

# Display the final file to the user
cat "$OUTPUT"
echo "==================================================="
```